

16-bit RISC Processor on FPGA

Complete Interview Guide: Theory, Design, Verilog, Vivado and Computer Organization

Project: Design and Implementation of a Basic 16-bit RISC Processor on the Boolean Board (Xilinx Spartan-7 XC7S50), Verilog HDL, Vivado

Prepared for Divyanshu, B.Tech ECE, NIT Durgapur

How to use this guide: read Part 1 to learn how to present the project, Parts 2-4 to be able to defend every design choice, Parts 5-6 for the Verilog and Vivado questions, Part 7 for limitations and improvements, and Part 8 for the Computer Organization and Architecture questions that interviewers attach to a processor project. The reference Verilog is a sketch that matches the specification; always describe **your own** design and numbers, and only claim what you actually built and measured.

Contents

Part 1: Project Overview and How to Present It

Part 2: Architecture and Datapath Deep Dive

Part 3: Module-by-Module Verilog (Reference Design)

Part 4: Test Program, Simulation and Verification

Part 5: Verilog HDL Concepts (Interview Q&A)

Part 6: Vivado, FPGA and Boolean Board (Interview Q&A)

Part 7: Design Decisions, Limitations and Improvements

Part 8: Computer Organization and Architecture Essentials

Part 1: Project Overview and How to Present It

Your 30-second pitch

I designed a 16-bit fixed-width RISC processor with an 8-bit datapath in Verilog HDL and deployed it on a Xilinx Spartan-7 (Boolean board) using Vivado. The custom ISA has nine instructions (NOP, ADD, SUB, MUL, CMP, MOV, LOAD, STORE, HALT) in two formats, eight 8-bit registers, and separate instruction and data memories (Harvard organisation). Execution is non-pipelined and multi-cycle: every instruction passes through Fetch, Decode, Read Operands, Execute, Memory and Write-Back, driven by a combinational control-unit decoder and a small stage sequencer. I simulated each module separately, then ran the mandatory test program in simulation and on the board, and read the registers and the Z/G/L flags on LEDs.

Be honest. If your own design differs from the reference in this guide (for example, you used a different stage counter or an asynchronous-read memory), describe your version. Interviewers go deep on whatever you claim, and a small, correct, well-understood design beats an impressive but half-understood one.

Specification at a glance

Parameter	Specification
Data width	8-bit (registers, ALU operands, memory data)
Instruction width	16-bit fixed
Registers	8 general-purpose, R0-R7, 8-bit each
Program Counter	increments by 1 each instruction; no branch or jump; freezes on HALT
Instruction memory	ROM / BRAM, preloaded with the program, addressed by PC (separate from data memory)
Data memory	512 x 8-bit RAM, 9-bit address, separate read and write enables
Execution model	sequential, non-pipelined; six stages of one clock each
Operations	NOP, ADD, SUB, MUL, CMP, MOV, LOAD, STORE, HALT
Flags	Z (equal), G (greater), L (less); updated by CMP only; one high at a time
Reset	synchronous, active-high; clears PC, IR, registers and flags
Mandatory modules	PC, Instruction Memory, IR, Register File, ALU, Multiplier, Data Memory, Control Unit, Top

Instruction formats and opcodes

Format	Fields (MSB to LSB)
R-type: ADD, SUB, MUL, CMP, MOV, NOP	opcode[15:12] rd[11:9] rs1[8:6] rs2[5:3] unused[2:0] = 000
M-type: LOAD, STORE	opcode[15:12] rd (LOAD) or rs (STORE) [11:9] address[8:0]

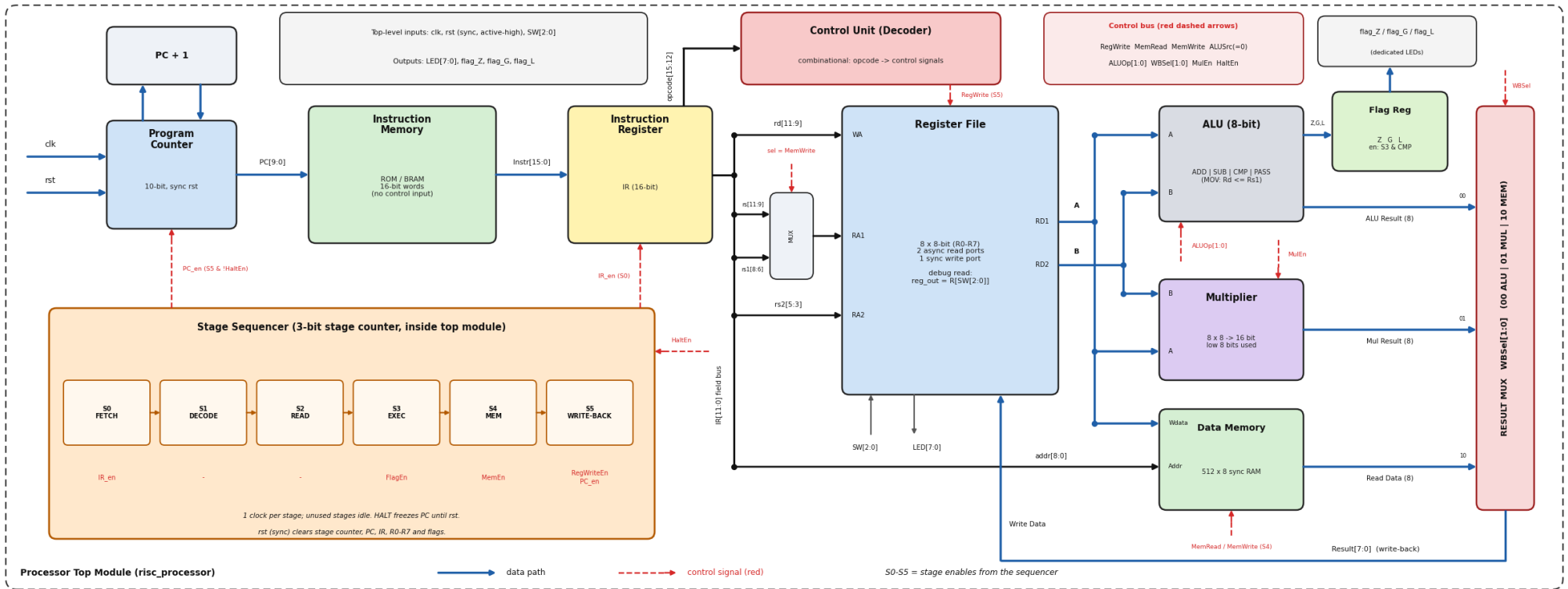
Opcode	Mnemonic	Operation
0000	NOP	no operation
0001	ADD	$Rd \leftarrow Rs1 + Rs2$ (8-bit unsigned, carry discarded)
0010	SUB	$Rd \leftarrow Rs1 - Rs2$ (8-bit unsigned, borrow discarded)
0011	MUL	$Rd \leftarrow$ low 8 bits of $(Rs1 \times Rs2)$, using a dedicated multiplier
0100	CMP	set Z, G, L from $Rs1$ versus $Rs2$; no register written
0101	MOV	$Rd \leftarrow Rs1$
0110	LOAD	$Rd \leftarrow MEM[address]$
0111	STORE	$MEM[address] \leftarrow Rs$
1111	HALT	freeze PC until reset

Reference datapath (corrected)

The next page shows the corrected datapath: the stage sequencer, the control signals (ALUOp[1:0], WBSel[1:0], MulEn, HaltEn), both operands feeding both the ALU and the multiplier, STORE data read through the RA1 multiplexer, and the debug path SW[2:0] to LED[7:0].

16-bit RISC Processor Datapath (corrected to the design specification)

8-bit data | 16-bit fixed instruction | 8 registers | non-pipelined, 6 clock cycles per instruction

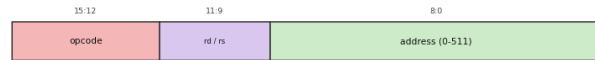


Instruction Formats (16-bit)

R-Type: ADD, SUB, MUL, CMP, MOV, NOP



M-Type: LOAD, STORE



Opcode Encoding

Opcode	Mnemonic	Operation
0000	NOP	no operation
0001	ADD	Rd <= Rs1 + Rs2
0010	SUB	Rd <= Rs1 - Rs2
0011	MUL	Rd <= (Rs1 x Rs2)[7:0]
0100	CMP	set Z, G, L (no write)
0101	MOV	Rd <= Rs1
0110	LOAD	Rd <= MEM[addr]
0111	STORE	MEM[addr] <= Rs
1111	HALT	freeze PC

6-Stage Execution Flow (every instruction, 1 clock per stage)

Stage	Action	Enable
S0 Fetch	IR <= InstrMem[PC]	IR_en
S1 Decode	Control Unit decodes opcode[15:12]	-
S2 Read Operands	RF reads Rs1, Rs2 (STORE: Rs from [11:9]); addr = IR[8:0]	-
S3 Execute	ALU / Multiplier operate; CMP updates Z, G, L	FlagEn
S4 Memory	LOAD reads, STORE writes Data Memory	MemEn
S5 Write Back	Rd <= Result (not for CMP/STORE/HALT); PC <= PC + 1	RegWriteEn, PC_en

Control Unit Truth Table (outputs of the decoder; stage enables gate them in time)

Instr	Opcode	RegWrite	MemRead	MemWrite	ALUSrc	ALUOp[1:0]	WBSe[1:0]	MulEn	HaltEn
NOP	0000	0	0	0	0	00	00	0	0
ADD	0001	1	0	0	0	00	00	0	0
SUB	0010	1	0	0	0	01	00	0	0
MUL	0011	1	0	0	0	00	01	1	0
CMP	0100	0	0	0	0	10	00	0	0
MOV	0101	1	0	0	0	11	00	0	0
LOAD	0110	1	1	0	0	00	10	0	0
STORE	0111	0	0	1	0	00	00	0	0
HALT	1111	0	0	0	0	00	00	0	1

What was corrected compared with the original diagram

- Control signals now match the spec: ALUOp[1:0], WBSe[1:0], MulEn, HaltEn, ALUSrc (was ALUOp[3:0], MemToReg, ...).
 - ALU and Multiplier both receive Rs1 and Rs2 (the original fed one operand to each).
 - STORE writes Rs (read via RA1 mux from [11:9]) - not the Result MUX output.
 - Data-memory address comes directly from IR[8:0]; ALU has a PASS so MOV works.
 - Z/G/L come from the ALU compare into a flag register, written only by CMP.
 - 3 write-back sources (ALU, MUL, MEM) need a 2-bit WBSe; 1-bit MemToReg cannot do it.
 - Added the 6-stage sequencer (S0-S5) that gates IR, flag, memory, register and PC enables.
 - Added SW[2:0] → LED[7:0] debug read, flag outputs and rst to PC, IR, RF, flags, sequencer.
- ALUOp: 00 ADD, 01 SUB, 10 CMP, 11 PASS(MOV). Unused outputs are 00 so FlagEn fires only on CMP.

Overview questions you will almost certainly get

Tell me about your project in two minutes.

Use a four-part structure: problem, design, verification, result.

- Problem: build a working processor from scratch to understand how instructions turn into hardware behaviour, targeting a real FPGA.
- Design: 16-bit fixed instructions, 8-bit datapath, eight registers, nine instructions, Harvard memories, multi-cycle six-stage execution with a decoder and a stage sequencer.
- Verification: unit testbenches per module, then a self-checking top-level testbench running the mandatory program and comparing registers, memory and flags against expected values.
- Result: the program leaves R1=6, R2=4, R3=10, R4=2, R5=40, MEM[20]=40 and G=1, shown on LEDs on the Boolean board; report your own resource and timing numbers from Vivado.

Example: Close with one sentence on learning: for example, how the load timing with synchronous RAM forced you to think about which stage the data is valid in.

What exactly did you design yourself, and what was given?

The specification (ISA, module list, test program, port names) was given. Your work is the microarchitecture and implementation: the datapath wiring, the control-signal table, the stage sequencing, every Verilog module, the testbenches, the constraints file, and the board bring-up. Say this plainly. If you used any code from the internet or from a classmate, say what you changed and what you understood; interviewers respect honesty and will test understanding either way.

What was the hardest part of the project?

Pick something real that you faced. Common genuine difficulties on this project:

- Getting the timing of LOAD right with a synchronous-read RAM: data is only valid one clock after the read enable, so write-back must wait.
- STORE needs the register named in bits [11:9], not rs1 in [8:6], so the register file read address needs a multiplexer.
- Making sure each instruction has exactly one side effect: register write, memory write, flag update and PC increment must each happen in one specific stage, not in every cycle.
- Keeping CMP flags one-hot and cleared by reset, and updating them only on CMP.
- Moving from simulation to the board: pin constraints, clock rate too fast to watch, button debouncing and reset polarity.

How do you know your processor is correct?

Layered verification. (1) Each module (ALU, multiplier, register file, decoder, memories) has a small testbench with directed cases, including boundary values. (2) The top-level testbench loads the mandatory program, waits for HALT, and automatically compares R1-R5, MEM[20] and the flags to expected values, printing PASS or FAIL. (3) Waveforms show fetch, decode and execute behaviour per instruction. (4) The same program is run on the board and the LEDs are compared with the expected binary values. (5) Extra tests: arithmetic wrap-around, CMP equal and less, STORE followed by LOAD, reset in the middle of execution, and HALT staying frozen.

Part 2: Architecture and Datapath Deep Dive

These are the questions an interviewer asks to check that you understand why the processor works, not just that it works.

Walk me through your processor's architecture.

Fetch side: the Program Counter addresses the Instruction Memory (ROM/BRAM); the 16-bit word is latched into the Instruction Register (IR). From the IR, opcode[15:12] goes to the Control Unit, which is a combinational decoder producing RegWrite, MemRead, MemWrite, ALUSrc, ALUOp[1:0], WBSel[1:0], MulEn and HaltEn.

Execute side: the register file reads two operands (Rs1, Rs2) at once. The ALU performs ADD, SUB, CMP and PASS; a separate multiplier produces the low 8 bits of Rs1 x Rs2. CMP results go into the Z/G/L flag register.

Memory side: the Data Memory (512 x 8) is addressed by IR[8:0]; LOAD reads it, STORE writes the register value to it.

Write-back: a result multiplexer chooses ALU, multiplier or memory data (WBSel) and writes it to Rd. A stage sequencer steps through six stages so that each side effect happens once per instruction.

Example: Draw this on paper in the interview: PC -> ROM -> IR -> (decoder, register file) -> ALU / multiplier -> result mux -> register file, with the data memory hanging off the side.

Why a 16-bit instruction when the data is only 8 bits?

Instruction width and data width are independent design choices. The instruction has to hold an opcode plus operands. R-type needs 4 (opcode) + 3 (rd) + 3 (rs1) + 3 (rs2) = 13 bits, with 3 spare. M-type needs 4 (opcode) + 3 (register) + 9 (address for 512 locations) = 16 bits. An 8-bit instruction could not fit an opcode, a register and a 9-bit address. Fixed 16-bit width also keeps fetch and decode simple, which is a classic RISC principle: one memory read per instruction, fields always in the same bit positions.

Example: Register fields are 3 bits because $2^3 = 8$ registers; the opcode is 4 bits, so up to 16 operations (9 used, space to extend).

Is this a Harvard or a von Neumann architecture? Why separate memories?

Harvard: instructions and data live in separate memories with separate address and data paths. The specification requires it, and it also fits the FPGA well: instruction memory is 16 bits wide while data memory is 8 bits wide, and each maps to its own block RAM. Benefits are no instruction/data conflict for the same memory port, independent widths, and the ability to fetch and access data in the same cycle (useful once pipelined). The downside is that you cannot treat code as data, so self-modifying code or loading programs into data memory is not possible. Von Neumann uses one memory and one bus and is simpler but creates the von Neumann bottleneck. Modern CPUs are 'modified Harvard': split L1 instruction and data caches over a unified main memory.

Trace each instruction through the datapath.

Every instruction runs S0 Fetch, S1 Decode, S2 Read, S3 Execute, S4 Memory, S5 Write-Back, one clock each. What differs is which units are active:

Instr	Control outputs	S3 Execute	S4 Memory	S5 Write-Back
ADD	RegWrite=1, ALUOp=00, WBSel=00	ALU: Rs1 + Rs2	idle	Rd <= ALU result
SUB	RegWrite=1, ALUOp=01, WBSel=00	ALU: Rs1 - Rs2	idle	Rd <= ALU result
MUL	RegWrite=1, MulEn=1, WBSel=01	multiplier: low 8 bits	idle	Rd <= multiplier result
CMP	ALUOp=10 (no RegWrite)	compare; Z/G/L latched	idle	no register write
MOV	RegWrite=1, ALUOp=11 (PASS)	ALU passes Rs1	idle	Rd <= Rs1

Instr	Control outputs	S3 Execute	S4 Memory	S5 Write-Back
LOAD	RegWrite=1, MemRead=1, WBSel=10	idle (address = IR[8:0])	RAM read of MEM[addr]	Rd <= memory data
STORE	MemWrite=1 (RA1 reads IR[11:9])	idle	MEM[addr] <= Rs	no register write
NOP	all zero	idle	idle	nothing written
HALT	HaltEn=1	-	-	sequencer freezes; PC frozen

In all cases $PC \leq PC + 1$ happens in S5 (except HALT).

How does the control unit work, and why do you also need a stage sequencer?

The control unit is purely combinational: the 4-bit opcode selects one row of the truth table, giving the control values for that instruction. It says what the instruction needs, not when. Without timing, RegWrite would be high in all six cycles, so the register would be overwritten five times with whatever was on the write-back bus (for LOAD, before the RAM output was valid), and the PC would increment repeatedly. The stage sequencer (a small 3-bit counter) supplies the 'when': IR loads only in S0, flags update only in S3, memory read/write only in S4, register write and PC increment only in S5. In the top module these are AND-ed: $we_reg = RegWrite$ and $stage == S5$.

Example: This is a hardwired control unit (logic gates), as opposed to a microprogrammed one that reads control words from a control store.

Single-cycle, multi-cycle and pipelined: which is yours, and what is its CPI?

Yours is multi-cycle with a uniform six cycles per instruction, so $CPI = 6$ for every instruction. The execution time of a program is $IC \times CPI \times T_{clk}$. The test program has 8 instructions; HALT is detected in its decode stage, so the program finishes in about $7 \times 6 + 2 = 44$ clocks, i.e. about 0.44 microseconds at 100 MHz.

Comparison: single-cycle has $CPI = 1$ but the clock must fit the slowest instruction (LOAD through every unit), so it is slow. Multi-cycle shortens the clock to the slowest stage and lets simple instructions use fewer cycles (yours does not, because the specification makes every instruction pass through all six stages). Pipelined overlaps stages to approach $CPI = 1$ with a short clock, at the cost of hazard logic.

Example: Idle stages are a deliberate simplification: the memory stage does nothing for ADD, but keeps control uniform and easy to verify.

How does HALT work?

The decoder asserts HaltEn when opcode = 1111. In the reference design the sequencer holds itself in the decode state while HaltEn is high, so S5 is never reached: the PC is never incremented, no register or memory write happens, and the IR keeps holding HALT, so the processor stays frozen until rst is asserted. Registers keep their values, so you can read the final state on the LEDs using SW[2:0]. On the board this is convenient: you can run at the full clock rate and still read the answer afterwards.

How is CMP implemented, and why is only one flag high at a time?

The ALU compares Rs1 and Rs2 with three parallel comparators: $Z = (a == b)$, $G = (a > b)$, $L = (a < b)$. For any pair of values exactly one is true, so the flags are one-hot by construction. They are stored in three flip-flops that are written only in S3 and only when the instruction is CMP; all other instructions leave them unchanged, and reset clears them. The comparison is unsigned because the operands are declared as unsigned vectors; for signed data you would compare with \$signed or use sign and overflow flags. CMP does not write any register.

Example: CMP R3, R4 with R3=10, R4=2 gives Z=0, G=1, L=0.

How does STORE get the register to write?

In M-type format there is no rs1/rs2 field. Bits [11:9] hold Rs, the register whose value is stored. The register file's first read port normally uses rs1 = IR[8:6], so a 2-to-1 multiplexer in front of RA1 selects IR[11:9] when the instruction is STORE (the decoder's MemWrite signal can be the select). The read value (RD1) goes to the Data Memory write-data input, and IR[8:0] is the address. A common wiring mistake is connecting the result bus to the memory write data, which stores the wrong value.

Why does the spec require a dedicated multiplier, and what happens to overflow?

Multiplication is much larger and slower than addition, so keeping it out of the ALU keeps the ALU small and fast and shows modular design. An 8 x 8 multiplier produces a 16-bit product, but only the low 8 bits are written to Rd, so overflow is silently discarded (arithmetic modulo 256). For example $20 \times 20 = 400 = 0x190$, and the register receives $0x90 = 144$. On Spartan-7, writing $a * b$ usually infers a DSP48E1 slice (or LUTs for small widths, depending on the synthesis settings); you can steer this with the `use_dsp` attribute. A sequential shift-and-add multiplier would save area at the cost of several cycles.

Example: Test boundary: $16 \times 17 = 272$, low 8 bits = 16.

How do ADD and SUB handle overflow, and are your numbers signed?

The spec defines unsigned 8-bit operation: carries and borrows beyond 8 bits are discarded, so results wrap modulo 256 ($200 + 100 = 44$, $3 - 5 = 254$). There are no carry or overflow flags, so software cannot detect wrap-around. The same adder works for two's-complement signed numbers (a property of two's complement), but then the programmer would interpret 254 as -2 and would need overflow detection. Z/G/L are unsigned comparisons.

Explain the LOAD timing with a synchronous-read RAM.

With a block RAM, the address is registered and the data appears at the output one clock edge after the read is enabled. In the reference timing, the address (IR[8:0]) is stable from decode, MemRead is asserted during S4, and the RAM output register is loaded at the end of S4. In S5 the result mux selects memory data (WBSel = 10) and the register file captures it at the end of S5. If the RAM were asynchronous (distributed LUT RAM), the data would be valid during S4 itself. The six-stage structure, with a dedicated memory stage before write-back, is what makes the synchronous RAM work without extra wait logic.

What happens if the instruction memory contains an undefined opcode?

Good practice is a default case in the decoder that drives every control output to zero, so an undefined opcode behaves as a NOP: nothing is written, the PC advances. This also prevents latches in synthesis because every output is assigned in every path. After reset the IR is cleared to 0000, which is NOP, so the first fetch starts from a harmless state.

Can R0 be hardwired to zero, and should it?

The spec says it may optionally be hardwired. Hardwiring R0 to zero (as in MIPS and RISC-V) gives a free constant, which with this ISA mostly helps MOV and comparisons against zero. But the mandatory program writes only R1-R5, so either choice passes; the important point is to state your choice and implement it consistently (writes to R0 ignored, reads return 0).

Part 3: Module-by-Module Verilog (Reference Design)

This is a reference sketch that follows the specification and the corrected datapath. Compare it with your own code, keep your naming where it differs, and make sure you can explain every line. Port names of the top module must match the spec (clk, rst, SW, LED, flag_Z, flag_G, flag_L).

Program Counter

```
module program_counter (input clk, rst, pc_en, output reg [9:0] pc);
    always @(posedge clk) begin
        if (rst) pc <= 10'd0; // synchronous, active-high reset
        else if (pc_en) pc <= pc + 10'd1; // only in write-back stage, never on HALT
    end
endmodule
```

Instruction Memory and Instruction Register

```
module instr_mem (input [9:0] addr, output [15:0] instr);
    reg [15:0] rom [0:1023];
    initial $readmemh("program.mem", rom); // or $readmemb with binary text
    assign instr = rom[addr]; // asynchronous read -> LUT ROM
endmodule

module instr_reg (input clk, rst, ir_en, input [15:0] d, output reg [15:0] ir);
    always @(posedge clk)
        if (rst) ir <= 16'h0000; // 0000... = NOP
        else if (ir_en) ir <= d; // load only in the Fetch stage
endmodule
```

program.mem for the mandatory program (one 16-bit hex word per line): 620A, 640B, 1650, 2850, 3AD0, 7A14, 40E0, F000.

Register File

```
module regfile (input clk, rst, we,
                input [2:0] ra1, ra2, wa, dbg_sel,
                input [7:0] wd,
                output [7:0] rd1, rd2, dbg_out);
    reg [7:0] r [0:7];
    integer i;
    always @(posedge clk) begin
        if (rst) for (i = 0; i < 8; i = i + 1) r[i] <= 8'd0;
        else if (we) r[wa] <= wd; // synchronous write, one port
    end
    assign rd1 = r[ra1]; // two asynchronous reads
    assign rd2 = r[ra2];
    assign dbg_out = r[dbg_sel]; // third read for SW[2:0] -> LED
endmodule
```

ALU, Flags and Multiplier

```
module alu (input [7:0] a, b, input [1:0] op,
            output reg [7:0] y, output z, g, l);
    always @(*) begin
        case (op)
            2'b00: y = a + b; // ADD
            2'b01: y = a - b; // SUB
            2'b10: y = 8'd0; // CMP: result unused
            2'b11: y = a; // PASS (MOV)
            default: y = 8'd0;
        endcase
    end
    assign z = (a == b);
    assign g = (a > b);
    assign l = (a < b);
endmodule

module multiplier (input [7:0] a, b, output [7:0] y);
    wire [15:0] p = a * b;
    assign y = p[7:0]; // low 8 bits of the 16-bit product
endmodule
```

```
endmodule
```

Data Memory

```
module data_mem (input clk, we, re, input [8:0] addr, input [7:0] wd,
                 output reg [7:0] rd);
    reg [7:0] mem [0:511];
    integer k;
    initial begin
        for (k = 0; k < 512; k = k + 1) mem[k] = 8'd0;
        mem[10] = 8'd6; mem[11] = 8'd4; // test data from the spec
    end
    always @(posedge clk) begin
        if (we) mem[addr] <= wd; // STORE
        if (re) rd <= mem[addr]; // LOAD (synchronous read -> BRAM)
    end
endmodule
```

Control Unit (decoder)

```
module control_unit (input [3:0] opcode,
                    output reg reg_write, mem_read, mem_write, alu_src, mul_en, halt_en,
                    output reg [1:0] alu_op, wb_sel);
    always @(*) begin
        // defaults: every output assigned on every path -> no latches
        reg_write = 0; mem_read = 0; mem_write = 0; alu_src = 0;
        mul_en = 0; halt_en = 0; alu_op = 2'b00; wb_sel = 2'b00;
        case (opcode)
            4'b0000: ; // NOP
            4'b0001: reg_write = 1; // ADD
            4'b0010: begin reg_write = 1; alu_op = 2'b01; end // SUB
            4'b0011: begin reg_write = 1; mul_en = 1; wb_sel = 2'b01; end // MUL
            4'b0100: alu_op = 2'b10; // CMP
            4'b0101: begin reg_write = 1; alu_op = 2'b11; end // MOV
            4'b0110: begin reg_write = 1; mem_read = 1; wb_sel = 2'b10; end // LOAD
            4'b0111: mem_write = 1; // STORE
            4'b1111: halt_en = 1; // HALT
            default: ; // undefined -> NOP
        endcase
    end
endmodule
```

Processor Top Module (sequencer, enables and interconnect)

```
module risc_processor (input clk, rst, input [2:0] SW,
                      output [7:0] LED, output flag_Z, flag_G, flag_L);
    localparam S_FETCH = 3'd0, S_DEC = 3'd1, S_READ = 3'd2,
               S_EXE = 3'd3, S_MEM = 3'd4, S_WB = 3'd5;
    reg [2:0] stage;
    wire [9:0] pc; wire [15:0] instr, ir; wire [3:0] opcode = ir[15:12];
    wire reg_write, mem_read, mem_write, alu_src, mul_en, halt_en;
    wire [1:0] alu_op, wb_sel;
    wire [7:0] rd1, rd2, alu_y, mul_y, dm_rd, wb_data, dbg_out;
    wire alu_z, alu_g, alu_l;
    reg fz, fg, fl;

    // ---- stage sequencer: HALT freezes it in the decode stage ----
    always @(posedge clk)
        if (rst) stage <= S_FETCH;
        else if (stage == S_DEC && halt_en) stage <= S_DEC;
        else stage <= (stage == S_WB) ? S_FETCH : stage + 3'd1;

    // ---- stage-gated enables ----
    wire ir_en = (stage == S_FETCH);
    wire flag_en = (stage == S_EXE) && (alu_op == 2'b10);
    wire we_mem = mem_write && (stage == S_MEM);
    wire re_mem = mem_read && (stage == S_MEM);
    wire we_reg = reg_write && (stage == S_WB);
    wire pc_en = (stage == S_WB) && !halt_en;

    program_counter u_pc (clk, rst, pc_en, pc);
    instr_mem u_imem (pc, instr);
    instr_reg u_ir (clk, rst, ir_en, instr, ir);
    control_unit u_cu (opcode, reg_write, mem_read, mem_write, alu_src,
                      mul_en, halt_en, alu_op, wb_sel);

    wire [2:0] ra1 = mem_write ? ir[11:9] : ir[8:6]; // STORE reads Rs from [11:9]
endmodule
```

```

regfile u_rf (clk, rst, we_reg, ra1, ir[5:3], ir[11:9], SW, wb_data,
             rd1, rd2, dbg_out);

alu      u_alu (rd1, rd2, alu_op, alu_y, alu_z, alu_g, alu_l);
multiplier u_mul (rd1, rd2, mul_y);
data_mem  u_dmem(clk, we_mem, re_mem, ir[8:0], rd1, dm_rd);

assign wb_data = (wb_sel == 2'b00) ? alu_y :
                 (wb_sel == 2'b01) ? mul_y : dm_rd;

always @(posedge clk)
    if (rst) {fz, fg, fl} <= 3'b000;
    else if (flag_en) {fz, fg, fl} <= {alu_z, alu_g, alu_l};

assign LED = dbg_out;
assign flag_Z = fz; assign flag_G = fg; assign flag_L = fl;
endmodule

```

Design notes interviewers like to hear

- Every always block is either purely combinational (always @(*), blocking assignments, every output assigned) or purely sequential (always @(posedge clk), non-blocking assignments).
- Resets are synchronous and active-high as required, and clear PC, IR, registers, flags and the stage counter. The data memory is initialised at configuration time, not by reset.
- The instruction memory is read asynchronously (small program, LUT ROM). If you want BRAM inference, register the output and add one fetch cycle; the six-stage structure absorbs it.
- ALUSrc exists only to match the specification's control list; with no immediates it is always 0.
- Control outputs are gated by stage enables rather than generating a slower clock, which keeps one clock domain.

Short module questions

Why are the register-file reads asynchronous but the write synchronous?

Reads must deliver both operands within the same cycle so that the ALU can use them in the Execute stage, so they are modelled as combinational multiplexers (a continuous assign). The write is a clocked event so that the register changes only on a clock edge, in one defined stage, which avoids glitches and read-after-write races inside a cycle. On the FPGA, an 8 x 8 register array maps to flip-flops or distributed RAM; with three read ports flip-flops plus multiplexers are used.

Why does the ALU use always @(*) and blocking assignments?

Because it is combinational logic: the output must update whenever any input changes, and the statements evaluate in order. always @(*) automatically includes all inputs in the sensitivity list, so there is no risk of forgetting one (which causes simulation and synthesis mismatch). Non-blocking assignments are reserved for sequential logic.

What would go wrong if the decoder did not assign every output in every case?

Verilog keeps the old value of any signal not assigned on some path, so synthesis infers a latch (level-sensitive storage) for that signal. Latches cause timing problems and unexpected behaviour. Giving all outputs default values at the top of the always block, and a default case, guarantees pure combinational logic.

Part 4: Test Program, Simulation and Verification

The mandatory program and its machine code

PC	Instruction	Binary	Hex	Effect
0	LOAD R1, 10	0110_001_000001010	620A	R1 <= MEM[10] = 6
1	LOAD R2, 11	0110_010_000001011	640B	R2 <= MEM[11] = 4
2	ADD R3, R1, R2	0001_011_001_010_000	1650	R3 <= 6 + 4 = 10
3	SUB R4, R1, R2	0010_100_001_010_000	2850	R4 <= 6 - 4 = 2
4	MUL R5, R3, R2	0011_101_011_010_000	3AD0	R5 <= 10 x 4 = 40
5	STORE R5, 20	0111_101_000010100	7A14	MEM[20] <= 40
6	CMP R3, R4	0100_000_011_100_000	40E0	10 > 2, so G=1, Z=0, L=0
7	HALT	1111_000_000_000_000	F000	PC frozen

Expected final state: R1=6 (0000_0110), R2=4 (0000_0100), R3=10 (0000_1010), R4=2 (0000_0010), R5=40 (0010_1000), MEM[20]=40, flag_G=1, flag_Z=0, flag_L=0.

Cycle-by-cycle view (what to show on the waveform)

Clocks	Instruction	Signals to point at
0-5	LOAD R1,10	stage 0..5; ir=620A; MemRead high in S4; dm_rd=6; we_reg in S5; R1 becomes 6; pc 0 -> 1
6-11	LOAD R2,11	ir=640B; dm_rd=4; R2 becomes 4; pc 1 -> 2
12-17	ADD R3,R1,R2	rd1=6, rd2=4, alu_y=10 in S3; wb_sel=00; R3 becomes 10
18-23	SUB R4,R1,R2	alu_op=01; alu_y=2; R4 becomes 2
24-29	MUL R5,R3,R2	mul_en=1; mul_y=40; wb_sel=01; R5 becomes 40
30-35	STORE R5,20	mem_write=1; ra1=ir[11:9]=5, so rd1=40; we_mem in S4; MEM[20]=40
36-41	CMP R3,R4	alu_op=10; flag_en in S3; {Z,G,L} = 010; no register written
42-43	HALT	halt_en=1; stage stays at decode; pc stays 7; nothing changes afterwards

Self-checking top-level testbench

```
`timescale 1ns/1ps
module tb_risc;
    reg clk = 0, rst = 1; reg [2:0] SW = 0;
    wire [7:0] LED; wire fz, fg, fl;
    integer errors = 0;

    risc_processor dut (.clk(clk), .rst(rst), .SW(SW), .LED(LED),
        .flag_Z(fz), .flag_G(fg), .flag_L(fl));
    always #5 clk = ~clk; // 100 MHz

    task check_reg(input [2:0] r, input [7:0] expected);
        begin
            SW = r; #1; // debug port selects register r
            if (LED !== expected) begin
                errors = errors + 1;
                $display("FAIL: R%d = %0d, expected %0d", r, LED, expected);
            end else $display("PASS: R%d = %0d", r, LED);
        end
    endtask

    initial begin
        repeat (3) @(posedge clk); rst = 0;
        repeat (60) @(posedge clk); // 8 instr x 6 clocks + margin
        check_reg(3'd1, 8'd6); check_reg(3'd2, 8'd4);
        check_reg(3'd3, 8'd10); check_reg(3'd4, 8'd2); check_reg(3'd5, 8'd40);
        if (dut.u_dmem.mem[20] !== 8'd40) begin errors = errors + 1; $display("FAIL: MEM[20]"); end
        if ({fz, fg, fl} !== 3'b010) begin errors = errors + 1; $display("FAIL: flags"); end
        if (errors == 0) $display("ALL TESTS PASSED");
    end
endmodule
```

```

        else                $display("%0d TEST(S) FAILED", errors);
        $finish;
    end
endmodule

```

Verification questions

How did you test the design, and what did you check besides the mandatory program?

Unit tests first, then integration. Per module: ALU (every ALUOp, boundaries like $255 + 1$, $0 - 1$), multiplier (255×255 low byte, 16×17), register file (write then read both ports, simultaneous read of the same register, reset clears), decoder (all nine opcodes plus an undefined one), memories (preload visible, write then read). Top level: the mandatory program with automatic checks, then extra programs: CMP with equal operands ($Z=1$) and $Rs1 < Rs2$ ($L=1$), STORE followed by LOAD of the same address, MOV, NOP in the middle of code, reset asserted mid-program and rerun, and a check that HALT leaves PC and registers unchanged for many cycles.

What is a self-checking testbench, and why is it better than looking at waveforms?

A self-checking testbench computes or stores the expected result and compares it with the DUT output automatically, printing PASS or FAIL. Waveform inspection is manual, slow and error-prone, and does not scale when you change the design: you would have to re-inspect everything. A self-checking bench becomes a regression test you can rerun after every edit. Use `!=` (case inequality) so that X or Z values count as failures rather than being silently treated as unequal-or-unknown.

Why `!=` instead of `!=` in the testbench?

`!=` compares logically; if either side contains X or Z the result is X, which an if statement treats as false, so a broken DUT outputting X might not trigger the error branch. `!=` compares four-state values exactly, so an X on the output is reported as a mismatch.

What is the difference between behavioural, post-synthesis and post-implementation simulation?

Behavioural (RTL) simulation runs your Verilog with no delays; it checks functionality. Post-synthesis functional simulation uses the synthesized netlist (LUTs and flip-flops), still without delays, and catches problems such as latches or simulation/synthesis mismatches. Post-implementation timing simulation back-annotates real routing and cell delays (SDF) and catches timing problems, though static timing analysis is the normal sign-off for timing. For this project, behavioural simulation plus the timing report plus the on-board test is the usual set.

What kinds of bugs are typical in a processor like this, and how do you find them?

- Wrong write timing: register written in every cycle or in the wrong stage; fix by AND-ing enables with the stage.
- STORE stores the wrong register or the wrong data; check `ra1` and the memory write data.
- Flags updating on non-CMP instructions, or not cleared by reset.
- LOAD getting stale data with synchronous RAM; check the stage in which MemRead is asserted and when the data is valid.
- Unknown (X) values in simulation: caused by missing reset or uninitialised memory; look at where the X first appears and trace backwards.
- Latches or multiple drivers from incomplete assignments.

How would you verify the Boolean-board result matches simulation?

Use `SW[2:0]` to select `R1..R5` and compare `LED[7:0]` with the expected binary values (`0000_0110`, `0000_0100`, `0000_1010`, `0000_0010`, `0010_1000`), and check the flag LEDs for $G=1$, $Z=0$, $L=0$. To read `MEM[20]`, add a short program that LOADs address 20 into a register and halts (this is how the spec expects memory to be read back). You can also add the Vivado ILA to capture pc, stage and ir live over JTAG.

Part 5: Verilog HDL Concepts (Interview Q&A)

What is the difference between blocking (=) and non-blocking (<=) assignments?

Blocking assignments execute in order: the right side is evaluated and the left side updated immediately, before the next statement runs. They model combinational logic. Non-blocking assignments evaluate all right-hand sides first and update the left sides together at the end of the time step, which models flip-flops that sample their inputs at the same clock edge. Rule of thumb: use <= in clocked always blocks and = in combinational always blocks; never mix them for the same variable in one block.

Example: Swap: always @(posedge clk) begin a <= b; b <= a; end swaps the values. With blocking = both become the old b.

Explain wire versus reg, and when to use assign versus always.

wire is a connection that must be driven continuously (by assign or a module output). reg is a variable that holds a value assigned inside a procedural block (always or initial). A reg does not necessarily become a flip-flop: always @(*) with a reg output creates combinational logic, while always @(posedge clk) creates flip-flops. Use assign for simple continuous expressions (multiplexers, comparators) and always @(*) for case or if-else logic.

What is a latch, how is it inferred accidentally, and how do you avoid it?

A latch is level-sensitive storage. It is inferred when a combinational always block does not assign a signal on every path, for example an if without else or a case without default; the synthesizer must remember the old value. Avoid it by assigning default values at the top of the block, covering all branches, and including a default case. Vivado reports inferred latches as warnings; treat them as bugs in a synchronous design.

Combinational versus sequential logic in Verilog: how do you write each?

Combinational: assign statements or always @(*) with blocking assignments and complete assignment. Sequential: always @(posedge clk) with non-blocking assignments, with a reset branch for state. Mixed designs separate the next-state logic (combinational) from the state register (sequential), as in two-block FSM style.

Synchronous versus asynchronous reset: which did you use and why?

Synchronous reset is checked only on a clock edge: always @(posedge clk) if (rst) ... Asynchronous reset is in the sensitivity list: always @(posedge clk or posedge rst). The specification requires a synchronous active-high reset. Advantages: purely synchronous design, easier timing analysis, immune to reset glitches. Disadvantages: the clock must be running, and the reset consumes logic in the data path (Xilinx flip-flops have dedicated set/reset pins, so the cost is small). Asynchronous resets are fast but must be released synchronously to avoid recovery/removal violations.

What does \$readmemb / \$readmemh do, and does it synthesize?

They load a memory array from a text file at simulation start (b for binary, h for hexadecimal text). Vivado supports them in synthesis for initialising ROM or RAM contents: the file is read at synthesis time and the values become the initial BRAM or LUT-ROM contents in the bitstream. The file must be added to the project (as a memory file or data file) so that both simulation and synthesis find it; a wrong path is a very common reason for an all-zero program.

What is an initial block, and is it synthesizable?

An initial block executes once at time zero. In testbenches it is used for stimulus. In synthesizable code, FPGA tools accept initial values for registers and memories because the FPGA configuration can set them (for example `reg [7:0] mem [0:511] initialised in an initial block`). Delays (`#10`) and most system tasks are ignored or unsupported in synthesis.

Which Verilog constructs are not synthesizable?

Delays (`#`), initial blocks used for behaviour other than initialisation, `force/release`, `fork-join`, `$display` and file I/O (ignored), real numbers, division and modulus by non-constants (supported poorly or very expensive), and loops with data-dependent bounds. Loops with constant bounds (for example the register reset loop) unroll and are fine.

Explain parameter, localparam and generate.

`parameter` defines a constant that can be overridden when instantiating a module (for example data width); `localparam` is a constant that cannot be overridden from outside, useful for state encodings like the stage numbers. `generate` (with `for` or `if`) creates repeated or conditional hardware at elaboration time, for example an array of identical cells. Parameterised modules make the design reusable: you could widen the datapath without rewriting the code.

What is the difference between case, casez and casex?

`case` compares exactly. `casez` treats `Z` (and `?`) in the case item as don't care; `casex` treats `X` and `Z` as don't care, which can hide bugs because an `X` in the input may match. Use `case` with a default for decoders; use `casez` only for priority patterns, and avoid `casex`.

How would you write an FSM in Verilog, and what is the difference between Moore and Mealy?

Use a state register (sequential block) plus a next-state function (combinational block) and an output block. In a Moore machine the outputs depend only on the current state; in a Mealy machine they depend on state and inputs, so Mealy can respond one cycle earlier and often needs fewer states but its outputs can glitch with input changes. Your stage sequencer is a Moore-style counter FSM: its outputs (stage enables) depend only on the state, with `HALT` acting as an input that holds the state. Encodings: binary, one-hot (fast on FPGAs, uses more flip-flops) or Gray.

What is a race condition in simulation, and how do you avoid it?

A race occurs when the result depends on the arbitrary order in which the simulator evaluates events that happen at the same time, for example a testbench driving an input with a blocking assignment exactly at the clock edge on which the DUT samples. Avoid by using non-blocking assignments in clocked logic, by driving testbench inputs slightly after the active edge (`#1` or on the negative edge), and by keeping each variable driven from only one always block.

How does a block RAM get inferred, and what is the difference from distributed RAM?

Vivado infers a block RAM from an array written and read in a clocked block with a registered (synchronous) read, large enough to justify the primitive. A read that is asynchronous (`assign out = mem[addr]`) maps to distributed RAM built from LUTs. Block RAM: dedicated 36 Kb blocks, synchronous read, good for large memories. Distributed RAM: uses LUTs, asynchronous read, good for small memories like a register file. Your 512 x 8 data memory with a synchronous read maps to a block RAM; the instruction ROM with an asynchronous read maps to LUTs.

Why is the multiplier a separate module and not part of the ALU?

The specification forbids it: the ALU must not multiply. A separate multiplier module is instantiated at the top and feeds the write-back mux with its own select (WBSel = 01) and enable (MulEn). This also lets the synthesis tool map it to a DSP slice independently, and keeps the ALU's critical path short.

What is the difference between a task and a function?

A function returns a single value, has no timing control and executes in zero simulation time; a task can have multiple outputs and may contain delays or event controls. Functions are used for calculations in synthesizable code; tasks are mostly used in testbenches, as in the check_reg task.

What is the sensitivity list, and what does always @(*) do?

The sensitivity list names the signals whose changes trigger the block. For combinational logic every input read must be in it, otherwise simulation behaves like a latch. always @(*) (or always_comb in SystemVerilog) automatically includes all signals read in the block, preventing mismatches between simulation and synthesis.

Part 6: Vivado, FPGA and Boolean Board (Interview Q&A)

The flow you followed, in order

Step	What you do in Vivado	What it checks or produces
1. Create project	choose part XC7S50-CSGA324 (Boolean board), add design sources, constraints, memory file	project structure
2. Behavioural simulation	add testbench as simulation source; run xsim; inspect waveforms	functional correctness
3. Elaboration (RTL analysis)	open the schematic	hierarchy and connectivity; catches wiring and width errors
4. Synthesis	converts RTL to LUTs, flip-flops, BRAM, DSP	utilization and warnings (latches, unused)
5. Implementation	optimise, place, route	timing summary (WNS, TNS, WHS), power, routed utilization
6. Bitstream generation	write the .bit file	configuration data
7. Hardware Manager	open target over USB-JTAG, program device	design running on the board

What is the difference between synthesis and implementation?

Synthesis translates your RTL into a netlist of FPGA primitives (LUTs, flip-flops, BRAM, DSP, carry chains) and does logic optimisation, but does not know where anything is placed. Implementation has three steps: `opt_design` (more optimisation), `place_design` (assign each primitive to a physical site) and `route_design` (connect them through the routing fabric). Accurate timing only exists after implementation, because wire delay depends on placement and routing.

What is a constraints (XDC) file and what must it contain for this project?

The XDC file tells Vivado how the top-level ports map to physical pins and what the clock is. At minimum: pin location and I/O standard for `clk`, `rst`, `SW[2:0]`, `LED[7:0]` and the three flags, plus a `create_clock` constraint for the clock period. Without the clock constraint the timing report is meaningless.

```
# Pin numbers are placeholders: copy the real ones from Real Digital's master XDC for the Boolean board.
set_property -dict {PACKAGE_PIN <clk_pin> IOSTANDARD LVCMOS33} [get_ports clk]
create_clock -period 10.000 -name sys_clk [get_ports clk] ;# 100 MHz (check the board manual)
set_property -dict {PACKAGE_PIN <pin> IOSTANDARD LVCMOS33} [get_ports rst]
set_property -dict {PACKAGE_PIN <pin> IOSTANDARD LVCMOS33} [get_ports {SW[0]}]
set_property -dict {PACKAGE_PIN <pin> IOSTANDARD LVCMOS33} [get_ports {LED[0]}]
set_property -dict {PACKAGE_PIN <pin> IOSTANDARD LVCMOS33} [get_ports flag_G]
```

Explain setup time, hold time and slack. What do WNS, TNS and WHS mean?

Setup time is how long data must be stable before the capturing clock edge; hold time is how long it must remain stable after it. Setup slack = (clock period - clock uncertainty) - (data path delay + setup time). Positive slack means timing is met. WNS (worst negative slack) is the slack of the worst setup path; TNS is the sum of all negative setup slacks; WHS is the worst hold slack. If WNS is negative the design may fail at that clock frequency; hold violations are normally fixed automatically by the router. Maximum frequency is roughly $1 / (\text{period} - \text{WNS})$ when WNS is positive.

Example: With a 10 ns clock and WNS = +4.2 ns, the critical path is about 5.8 ns, so Fmax is around 170 MHz (approximate; use your own report).

What is the critical path in your processor, and how would you shorten it?

Look at your own timing report, but the likely candidates are: register file read -> multiplier (or ALU) -> result mux -> register file write, or the 8 x 8 combinational multiplier alone. Fixes: pipeline the multiplier (register inputs or outputs), use a sequential shift-add multiplier over several cycles, force the multiplier into a DSP slice, add a register after the read ports, or lower the clock. Because the design is multi-cycle, you could also declare a multicycle path in the constraints, though in the simple design the multiplier output is captured in the same cycle.

What resource utilization would you expect, and what do LUT, FF, BRAM and DSP mean?

LUT: look-up table implementing any 6-input logic function. FF: flip-flop storing one bit. BRAM: 36 Kb block RAM. DSP: DSP48E1 slice with a multiplier-accumulator. For this tiny design expect a few hundred LUTs and flip-flops, at most one DSP (or none if the multiplier maps to LUTs), and about one or two BRAMs (or none if memories map to LUT RAM). The XC7S50 has on the order of 32.6k LUTs, 65.2k flip-flops, 75 BRAM36 and 120 DSP slices (confirm in the Xilinx 7-series/Spartan-7 datasheet). Always quote the numbers from your own utilization report.

How did the memories map to FPGA resources?

The 512 x 8 data memory (4 Kb) with a synchronous read maps to a block RAM (or half of one); the 1024 x 16 instruction memory (16 Kb) maps to BRAM if read synchronously, or to LUT ROM if read asynchronously and small. The register file (8 x 8 = 64 bits, three read ports) uses flip-flops and multiplexers. Check the synthesis report's 'RAM' and 'ROM' inference messages, or the utilization report by resource type.

Why can you not simply run the processor at 100 MHz and watch the LEDs?

At 100 MHz the whole test program finishes in under a microsecond, far faster than the eye. Because HALT freezes everything, the final values remain on the LEDs, so you can read the result afterwards using SW[2:0]. To watch execution step by step, either use a slow clock-enable pulse from a counter (not a divided clock used as a clock, which creates clock-domain and skew issues) or a debounced single-step button, and display PC and stage on the 7-segment display or LEDs. Alternatively use the Vivado ILA to capture signals at full speed.

How do you handle a push button as reset or clock?

Mechanical buttons bounce for a few milliseconds, and they are asynchronous to the FPGA clock. Pass them through a two-flip-flop synchroniser (to avoid metastability), then a debounce counter (accept a new level only after it is stable for around 10 ms), and for a step button a one-pulse (edge-detect) circuit. For a plain reset, the synchroniser is the key part, and reset polarity must match the active-high rst in the spec; check whether the board button is active-high or active-low.

What is metastability, and how does a two-flop synchroniser help?

When an asynchronous input changes near a clock edge it can violate setup/hold, leaving the flip-flop output at an undefined voltage for a short time before it resolves to 0 or 1. Feeding the signal through two flip-flops in series gives the first one a full clock period to resolve, making the probability of the second flip-flop seeing an unresolved value extremely small (mean time between failures grows exponentially with the resolution time). Switches, buttons and any signal from another clock domain need this.

What is the Vivado ILA, and how could it be used here?

The Integrated Logic Analyzer is a debug core inserted into the design that captures internal signals in block RAM when a trigger condition occurs, and displays them in the Hardware Manager over JTAG. You would probe pc, stage, ir, register write enable and wb_data, and trigger on a specific PC value. It uses extra BRAM and LUTs and needs a clock; the VIO core is the companion for driving inputs from the PC.

What are common Vivado warnings and critical warnings, and which matter?

Matter: 'inferred latch', 'multi-driven net', 'unconnected port', 'timing constraints not met', 'no clock defined', 'unconstrained ports'. Usually harmless: unused-signal or trimmed-register warnings for logic that is genuinely not needed. A 'critical warning' about missing I/O standards or location constraints must be resolved before generating a bitstream.

What is the difference between an FPGA and a microcontroller, and why implement a processor on an FPGA?

A microcontroller is a fixed processor with peripherals executing software sequentially; an FPGA is a reconfigurable fabric where you define the hardware itself, so you can run many operations in parallel with deterministic timing. Implementing a processor on an FPGA is a learning exercise in computer architecture (ISA, datapath, control) and the basis of soft cores (MicroBlaze, RISC-V) used when you need custom instructions or peripherals next to a processor.

What is a clock constraint, and what happens if you forget it?

`create_clock` tells the timing engine the clock period (and waveform) so it can check setup and hold on every path. Without it, Vivado reports the paths as unconstrained and does not optimise for timing, so the design may pass implementation but fail on the board at higher frequencies.

What does the bitstream contain, and how does the FPGA get configured?

The bitstream holds the configuration bits for every LUT, routing switch, flip-flop initial value and BRAM content (including your preloaded program and data). SRAM-based FPGAs lose it at power-off, so it is loaded over JTAG from Vivado for testing, or from external flash at power-on for a permanent design (the Boolean board has JTAG and PROM jumper options).

Static timing analysis versus simulation: which sign-off do you trust?

Static timing analysis (STA) checks every timing path against the constraints without needing test vectors, so it covers all paths, while simulation only covers the paths your stimulus exercises. Functional simulation proves correctness of behaviour; STA proves that the implemented design meets the clock. Both are needed.

Part 7: Design Decisions, Limitations and Improvements

Interviewers often move from 'what you built' to 'what would you change'. Show that you know where the design is limited and how a real processor addresses each limit.

Limitation in your design	Why it exists	How to fix it
No branch or jump	spec removes control flow; PC only increments	add a branch/jump format, PC mux (PC+1, target, register), compare-and-branch using Z/G/L; add a link register for calls
No immediate operands	all arithmetic uses registers	add an I-type format with a sign-extended immediate and use ALUSrc to choose between Rs2 and the immediate (this is the real job of ALUSrc)
Only 8-bit data, 512 bytes	spec fixes widths	parametrise widths; add byte/word access; larger BRAM
No carry or overflow flags	flags are CMP only	add C, V, N flags and conditional branches (BEQ, BNE, BGT ...)
CPI = 6 for everything	uniform six-stage flow	variable cycles per instruction (skip unused stages), or pipeline to approach CPI = 1
No pipeline	sequential by spec	5-stage pipeline with hazard detection and forwarding
No subroutines, stack or interrupts	tiny ISA	stack pointer, CALL/RET, interrupt controller with saved PC
Program fixed at synthesis	ROM preloaded	UART bootloader writing a RAM instruction memory
No division, logic or shift ops	limited ISA	add AND, OR, XOR, shifts to the ALU (more ALUOp bits)

How would you turn this into a pipelined processor?

Split the six stages into pipeline stages (IF, ID, EX, MEM, WB; merge READ into ID) and insert pipeline registers between them (IF/ID, ID/EX, EX/MEM, MEM/WB). The control signals travel along with the instruction. Then handle hazards:

- Structural: separate instruction and data memories (already Harvard) avoid the IF/MEM conflict; the register file writes in the first half and reads in the second half or has bypass.
- Data hazards (RAW): add a forwarding unit that sends the EX/MEM or MEM/WB result to the ALU inputs; a load followed immediately by a dependent instruction needs a one-cycle stall (load-use hazard), since the data arrives only after MEM.
- Control hazards: once branches exist, flush wrong-path instructions, or predict.

How would you add branch instructions?

Add a B-type format: opcode, condition (or use the flags), and a target field. Add a PC multiplexer selecting PC + 1 or the branch target, controlled by PCSrc = Branch AND (condition from Z, G, L flags). For example BEQ jumps when Z = 1, BGT when G = 1, BLT when L = 1; this is why CMP sets three flags. A jump simply forces PCSrc = 1. In the pipelined version branches create control hazards, handled by stalling, flushing, or branch prediction. Note the spec forbids branches in this assignment, so this is a 'what would you do next' answer.

Why does the spec include ALUSrc if there are no immediates?

ALUSrc normally selects whether the second ALU operand comes from the register file or from a sign-extended immediate. With register-only operands it is constant 0, so it exists as a placeholder for extension (immediate instructions such as ADDI). Say so openly: it is generated by the decoder to match the interface, but has no effect in the base ISA.

What trade-offs did you make in this design?

Simplicity versus performance: uniform six-cycle execution gives simple control and easy verification, at the cost of CPI = 6. Area versus speed: a combinational multiplier is fast but large; a sequential multiplier would be small but slower. Registers versus memory: eight registers keep the register-file at 3 bits and the instruction fits 16 bits. Asynchronous ROM read versus BRAM: simpler fetch versus lower LUT use. Single clock domain with enables versus derived clocks: easier timing at the cost of a few multiplexers. Being able to name these trade-offs is more convincing than any single number.

How would you extend the processor to run a real program, such as computing a factorial or sorting?

You need control flow (branches), immediates or constants in memory, and more ALU operations. Add compare-and-branch using the existing flags, a jump instruction, an ADDI-style immediate, and perhaps a shift. Then write an assembler (a small Python script that turns mnemonics into the hex words in program.mem), so programs are not hand-encoded.

How would you add an interrupt?

Add an interrupt-request input synchronised to the clock. At an instruction boundary (end of S5) if enabled and requested, save the PC into a dedicated register, load the PC with the interrupt vector address, and disable further interrupts; a return-from-interrupt instruction restores the PC. Also save flags if they are shared. This requires branches/PC load, which the base design lacks.

How would you reduce power or area in this design?

Clock gating is avoided on FPGAs; use clock enables. Share hardware: one adder for ALU and PC increment is possible but adds multiplexers. Replace the combinational multiplier with a shift-and-add unit. Use block RAM for the register file only if it is bigger. In real ASIC design: operand isolation, clock gating, smaller memories. On FPGA, power is dominated by static power and clock tree, and a design this small uses very little.

What would you do differently if you started again?

Pick two or three concrete improvements you believe in: write the testbench before the RTL for each module, write a tiny assembler, add a debug interface (PC and stage on the 7-segment display), parameterise the widths, document the stage timing diagram first, and use the Vivado ILA earlier. This shows reflection and engineering maturity.

Part 8: Computer Organization and Architecture Essentials

Interviewers use a processor project as a gateway to COA. Each answer below ties back to your design where possible.

A. Architecture and instruction set

Compare Von Neumann and Harvard architectures.

Von Neumann: one memory holds instructions and data, one bus; simple and flexible (programs can be loaded and modified like data) but instruction fetch and data access compete for the bus (the von Neumann bottleneck). Harvard: separate instruction and data memories and buses, so fetch and data access can overlap and widths can differ; less flexible. Modern processors are modified Harvard: separate L1 instruction and data caches over unified main memory. Your processor is Harvard (16-bit instruction ROM, 8-bit data RAM).

RISC versus CISC.

RISC: fixed-length simple instructions, load/store architecture (only LOAD and STORE access memory), many registers, few addressing modes, hardwired control, easy pipelining; one instruction per cycle is the goal. CISC: variable-length complex instructions that can operate directly on memory, many addressing modes, often microprogrammed control, smaller code size but harder to pipeline. Examples: ARM, MIPS, RISC-V are RISC; x86 is CISC externally (and translates to RISC-like micro-operations internally). Your processor follows RISC principles: fixed 16-bit instructions, register operands for arithmetic, memory only by LOAD/STORE.

Classify ISAs by operand storage: stack, accumulator, register-memory, load-store.

Stack: operands are implicit on top of a stack (PUSH/POP, ADD), compact code. Accumulator: one operand is an implicit accumulator register. Register-memory: instructions can take one memory operand (x86). Load-store (register-register): arithmetic uses only registers, memory access by explicit load and store (MIPS, ARM, RISC-V, and your design). Three-address instructions name destination and two sources (as ADD Rd, Rs1, Rs2 does); two-address overwrite one source; one-address use the accumulator.

What are addressing modes? Which one does your processor use?

An addressing mode specifies how the operand address or value is obtained: immediate (value in the instruction), register direct (operand in a register), direct/absolute (address in the instruction), indirect (address stored in a register or memory), register indirect, indexed or base + displacement, relative (PC + offset), auto-increment/decrement, stack. Your processor uses register direct for arithmetic and direct (absolute) addressing for LOAD/STORE, where the 9-bit address is in the instruction. There is no immediate or indexed mode.

Explain the instruction cycle and the registers involved (PC, IR, MAR, MDR).

Fetch: $MAR \leq PC$; $MDR \leq Memory[MAR]$; $IR \leq MDR$; $PC \leq PC + 1$. Decode: the control unit interprets the opcode. Execute: operands are read, the ALU or memory acts, the result is written back. Interrupt check follows in some designs. In your processor, PC addresses the instruction memory directly, so there is no separate MAR/MDR for instructions (the memory output is captured in the IR), and the data memory address comes straight from IR[8:0]. Other special registers: stack pointer, status/flag register (your Z/G/L), program status word.

Hardwired versus microprogrammed control unit.

Hardwired: control signals are generated by combinational logic (gates, PLA, decoder) from the opcode and state: very fast, but hard to modify and complex for large ISAs; typical in RISC. Microprogrammed: each instruction runs a sequence of micro-instructions fetched from a control store (ROM); easier to design and change, suited to complex CISC instructions, but slower because of the extra control-store access. Horizontal microcode has wide words (more parallelism), vertical is encoded and narrower. Your design is hardwired: a decoder plus a stage counter.

What is the difference between instruction set architecture and microarchitecture?

The ISA is the programmer-visible contract: instructions, registers, addressing, data types, memory model. The microarchitecture is the implementation: pipeline, caches, execution units, control. Many microarchitectures can implement one ISA (Intel and AMD both implement x86). In your project, the ISA is the instruction table and encoding; the microarchitecture is the six-stage sequential datapath.

B. Performance and pipelining

State the CPU performance equation and use it.

CPU time = Instruction count $\times$ CPI $\times$ Clock period = $(IC \times CPI) / f$. Performance improves by reducing IC (better ISA, compiler), CPI (better microarchitecture) or the clock period (technology, pipelining). With a mix of instruction classes, $CPI = \text{sum of (fraction} \times CPI) \text{ over classes}$. $MIPS = f / (CPI \times 10^6)$ but is a poor comparison metric across ISAs.

Example: Worked: a 2 GHz CPU runs 10^9 instructions; 50% have CPI 1, 30% CPI 2, 20% CPI 4. $CPI = 0.5 + 0.6 + 0.8 = 1.9$. Time = $10^9 \times 1.9 / (2 \times 10^9) = 0.95$ s.

Explain instruction pipelining and the speedup formula.

Pipelining overlaps the execution of different instructions in different stages, like an assembly line. For k stages and n instructions with equal stage time, the pipelined time is $(k + n - 1)$ cycles versus $n \times k$ cycles unpipelined, so speedup = $n k / (k + n - 1)$, approaching k as n grows. Real speedup is lower due to hazards, unbalanced stages and pipeline-register overhead. Throughput improves; latency of a single instruction does not.

Example: Worked: 5 stages, 100 instructions: pipelined = $5 + 99 = 104$ cycles; unpipelined = 500; speedup = $500/104 = 4.81$.

What are pipeline hazards and how are they resolved?

Structural hazard: two instructions need the same hardware in the same cycle (for example one memory port for fetch and data); solved by duplicating resources (separate I and D memories). Data hazard: an instruction needs a result not yet written; types are RAW (true dependency), WAR and WAW (name dependencies, relevant for out-of-order); solved by forwarding/bypassing, stalling (inserting bubbles, needed for load-use), or compiler scheduling. Control hazard: the next instruction is unknown until a branch resolves; solved by stalling, flushing, delayed branches or branch prediction (static or dynamic with a branch history table).

Example: Example RAW: ADD R1,R2,R3 then SUB R4,R1,R5. Forward R1 from the ALU output to the SUB's input instead of waiting for write-back.

What is branch prediction?

A technique to guess the outcome and target of a branch early so the pipeline can keep fetching. Static: always not-taken, or backward-taken/forward-not-taken. Dynamic: a branch history table of 1-bit or 2-bit saturating counters per branch; the two-bit counter tolerates one anomaly (loop exit). Modern predictors use correlation and history (gshare, TAGE) with a branch target buffer. A misprediction costs the pipeline depth in flushed cycles.

Amdahl's law.

Overall speedup = $1 / ((1 - f) + f / s)$, where f is the fraction of execution time that is enhanced and s is the speedup of that fraction. It shows diminishing returns: the unenhanced part limits total improvement to $1 / (1 - f)$.

Example: Worked: 40% of the time is sped up 4 times: speedup = $1 / (0.6 + 0.1) = 1.43$. Even with infinite s the limit is $1 / 0.6 = 1.67$.

Superscalar, VLIW and other forms of parallelism (Flynn's taxonomy).

Instruction-level parallelism: superscalar processors issue several instructions per cycle using hardware dependency checking (and often out-of-order execution and register renaming); VLIW packs independent operations into one long instruction scheduled by the compiler. Flynn's taxonomy: SISD (classic uniprocessor), SIMD (vector/GPU: one instruction on many data), MISD (rare), MIMD (multicore, clusters). Your processor is SISD, scalar, in-order, non-pipelined.

C. Arithmetic and number representation

Explain two's complement, carry versus overflow, and sign extension.

Two's complement represents negative numbers by inverting the bits and adding one; an n-bit number ranges from -2^{n-1} to $2^{n-1} - 1$ and addition/subtraction use the same adder. Carry out of the MSB indicates unsigned overflow; signed overflow occurs when two operands of the same sign give a result of the opposite sign (equivalently carry into the MSB differs from carry out of the MSB). Sign extension copies the sign bit into the added upper bits to preserve value when widening.

Example: Example: 8-bit 0111_1111 (127) + 0000_0001 gives 1000_0000, which is -128 as signed: signed overflow. As unsigned it is 128, correct, with no carry.

How do adders work, and what is a carry-lookahead adder?

A full adder computes $\text{sum} = a \oplus b \oplus \text{cin}$ and $\text{cout} = ab + \text{cin}(a \oplus b)$. A ripple-carry adder chains n full adders, so delay grows linearly with n. A carry-lookahead adder computes generate ($g = ab$) and propagate ($p = a \oplus b$) for each bit and forms carries in parallel: $c_{i+1} = g_i + p_i c_i$, expanded across bits, so delay grows roughly logarithmically at the cost of more gates. On Xilinx FPGAs the synthesizer uses dedicated fast carry chains for a + b, so you rarely write adders by hand.

Explain shift-and-add multiplication and Booth's algorithm.

Shift-and-add: for each multiplier bit, add the multiplicand (shifted) to a partial product if the bit is 1; an n-bit multiplication needs n steps. Booth's algorithm recodes the multiplier to handle two's-complement numbers directly and to skip runs of ones: examine bit pairs (current, previous): 01 means add the multiplicand, 10 means subtract it, 00 and 11 mean only shift. It reduces additions for long runs of ones; radix-4 Booth halves the number of partial products. An array multiplier generates all partial products in parallel with a tree of adders (fast, large), which is what a DSP slice implements. Your MUL keeps only the low 8 bits of the 16-bit product.

Explain IEEE 754 single-precision format.

32 bits: 1 sign bit, 8 exponent bits (bias 127) and 23 fraction bits with an implied leading 1 for normalised numbers: $\text{value} = (-1)^s \times 1.f \times 2^{(e-127)}$. Special values: exponent all zeros = zero or denormals, exponent all ones = infinity or NaN. Double precision has 11 exponent bits (bias 1023) and 52 fraction bits. Floating-point addition needs exponent alignment, mantissa addition, normalisation and rounding.

Example: Example: $5.75 = 101.11$ binary = 1.0111×2^2 ; exponent = 129 = 1000_0001; fraction = 0111 followed by zeros; hex = 40B80000.

D. Memory system

Describe the memory hierarchy and the principle of locality.

From fast and small to slow and large: registers, L1/L2/L3 caches (SRAM), main memory (DRAM), secondary storage. It works because of locality: temporal locality (recently used items are reused) and spatial locality (nearby addresses are used soon). Caches keep recently used blocks close to the processor. SRAM uses six transistors per bit, is fast and expensive; DRAM uses one transistor and a capacitor, is dense and cheap but needs refresh. On the FPGA the equivalents are registers/LUT RAM (fast) and BRAM.

Explain cache mapping: direct, fully associative and set-associative.

Direct mapped: each memory block maps to exactly one cache line (index = block mod lines); simple and fast but suffers conflict misses. Fully associative: a block can go in any line; fewest conflicts but needs comparison with all tags (expensive). Set-associative (n-way): a block maps to one set and can occupy any of n lines in it; a compromise (2-way to 8-way typical). Address split: tag | index | block offset; offset bits = $\log_2(\text{block size})$, index bits = $\log_2(\text{number of sets})$.

Example: Worked: 32 KB direct-mapped cache, 64-byte blocks, 32-bit addresses: offset = 6 bits; lines = $32768/64 = 512$, so index = 9 bits; tag = $32 - 9 - 6 = 17$ bits.

AMAT, write policies, replacement and the three Cs of misses.

Average memory access time = hit time + miss rate x miss penalty (for multi-level: L1 hit time + L1 miss rate x (L2 hit time + L2 miss rate x memory penalty)). Write policies: write-through (write to cache and memory; simple, more traffic, often with a write buffer) and write-back (write only to cache, mark dirty, write on eviction; less traffic); on a write miss, write-allocate or no-write-allocate. Replacement: LRU, FIFO, random. Misses: compulsory (first access), capacity (cache too small), conflict (mapping collisions); a fourth, coherence, appears in multiprocessors.

Example: Worked: hit time 1 ns, miss rate 5%, miss penalty 20 ns: $\text{AMAT} = 1 + 0.05 \times 20 = 2$ ns.

What is virtual memory, paging and the TLB?

Virtual memory gives each process its own large address space, mapped to physical memory by the OS and hardware in fixed-size pages (for example 4 KB). A page table maps virtual page numbers to physical frames; if a page is not in memory, a page fault occurs and the OS loads it from disk. Because every access would need an extra memory read for the page table, a translation lookaside buffer (TLB), a small cache of recent translations, makes translation fast. Benefits: protection, isolation, programs larger than physical memory. Your processor has no virtual memory: addresses are physical and tiny.

E. I/O and system

Compare programmed I/O, interrupt-driven I/O and DMA.

Programmed I/O (polling): the CPU repeatedly checks a device status register; simple but wastes CPU time. Interrupt-driven: the device signals when ready; the CPU saves its state, runs an interrupt service routine (ISR) and returns; efficient for sporadic events. DMA: a controller transfers blocks between device and memory without CPU involvement, interrupting only on completion; best for high-volume transfers. Memory-mapped I/O places device registers in the memory address space (use ordinary LOAD/STORE); port-mapped I/O uses separate I/O instructions and address space.

Interrupt versus exception (trap), and how is an interrupt handled?

An interrupt is asynchronous, caused by external hardware (timer, keyboard); an exception is synchronous, caused by the executing instruction (divide by zero, illegal opcode, page fault). Handling: finish the current instruction, save PC and status, disable or mask lower-priority interrupts, jump to the handler via the vector table, run the ISR, restore state and return (RTI). Priority can be handled by daisy chain or a priority controller.

What is endianness?

The order in which the bytes of a multi-byte value are stored in memory. Big-endian stores the most significant byte at the lowest address; little-endian stores the least significant byte first (x86, most ARM systems). It matters for data exchange and byte-addressed memory; your 8-bit data memory avoids the issue entirely.

Example: Example: 0x12345678 at address 0x100: little-endian stores 78 56 34 12 starting from 0x100.

What is a bus, and what are the types of buses?

A bus is a shared set of wires that carries information between components. The address bus carries addresses (width sets the addressable space: 9 bits gives 512 locations in your data memory), the data bus carries data (8 bits in your processor), and the control bus carries signals such as read, write and clock. Buses can be synchronous or asynchronous, and arbitration decides which master may use a shared bus.

F. Quick numbers for your project

Item	Value / formula
Registers	$8 = 2^3$, so 3-bit register fields
Data memory	$512 \times 8 \text{ bits} = 4096 \text{ bits} = 4 \text{ Kb}$; address = $\log_2(512) = 9 \text{ bits}$
Instruction ROM (reference)	$1024 \times 16 \text{ bits} = 16 \text{ Kb}$ (10-bit PC); the program needs only 8 words
Opcode space	4 bits = 16 codes, 9 used
CPI	6 for every instruction (six stages, one clock each)
Test program length	8 instructions, about 44 clocks ($7 \times 6 + \text{HALT fetch and decode}$)
Time at 100 MHz	about 0.44 microseconds
MUL range	8×8 gives 16 bits; only [7:0] kept; modulo 256
Wrap-around examples	$200 + 100 = 44$; $3 - 5 = 254$; $20 \times 20 = 144$
Flags	Z, G, L one-hot, written only by CMP in the Execute stage

G. Rapid-fire one-liners

- Why 3-bit register fields? Eight registers. Why 9-bit address? 512 locations.
- What is the first thing after reset? PC = 0, stage = Fetch, IR = NOP.
- Which signal makes STORE read Rs from bits [11:9]? MemWrite selecting the RA1 mux.
- Which stage writes registers? Write-Back (S5), gated by the stage enable.
- Which module may multiply? Only the dedicated multiplier, not the ALU.
- What stops the processor? HALT; only rst restarts it.
- Is the design pipelined? No; multi-cycle, CPI = 6.
- Harvard or von Neumann? Harvard.
- Hardwired or microprogrammed control? Hardwired.
- Load/store architecture? Yes: memory only through LOAD and STORE.
- Why synchronous reset? Single clock domain, simple timing, required by the spec.